DevForge — 10-Week Full-Stack Developer Curriculum

> Internal curriculum document. Version: May 2025.
>
> This covers what students learn, what they build, and what they submit each week.

Program Philosophy

Students learn by building, not by watching. Every week produces something real — a commit, a pull request, a working feature, a deployed app. By the end of Week 10, a student has three deployed projects, 65+ merged pull requests, and a GitHub profile that answers the question "show me something you've built."

The stack is chosen deliberately: JavaScript → Node.js → React, in that order. Foundations first. Frameworks second. No skipping steps.

Jira tickets are introduced in Week 1 and used throughout the entire program. Students learn to read a ticket, break it into tasks, write a branch name from it, and close it with a PR — the same workflow used at every product company in India.

At a Glance

Week	Focus	What They Build
1	Git, JavaScript, APIs, Jira	CLI tools, API scripts, first PR
2	Node.js, Express, PostgreSQL, Prisma	REST API with full CRUD
3	React	Frontend connected to their own backend
4	Authentication + Project 1 Kickoff	JWT auth system + Project 1 setup
5	Project 1 — Core Features	SaaS app core (file uploads, CRUD)
6	Project 1 — Polish + Cloudinary	File storage, UI polish, Project 1 deployed
7	Project 2 — Payments + Real-time	Razorpay integration
8	Project 2 — Real-time + Complete	Socket.io, Project 2 deployed
9	Project 3 — Production Deployment	CI/CD, env config, 3rd app deployed
10	Career Week	Resume, LinkedIn, mock interviews, job strategy

---

Week 1 — Git, JavaScript, APIs & The Developer Workflow

Goal: By the end of Week 1, students are comfortable with the terminal, Git, JavaScript fundamentals, making API calls, and the pull request workflow. They understand what "frontend" and "backend" mean at a practical level.

Day 1 — Terminal and Git

Concepts:

- What the terminal is and why developers use it
- Navigating directories: `ls`, `cd`, `mkdir`, `touch`, `rm`
- Git as version control: why it exists, what a repository is
- The Git workflow: `init → add → commit → push`
- Setting up a GitHub account and SSH key
- Cloning a repo, making changes, pushing a commit

What students do:

- Create their first Git repository

- Write a `README.md` and commit it
- Push to GitHub
- Open their first pull request against the DevForge starter repo

Jira intro: Students are introduced to the DevForge Jira project. Each day's work has a pre-created Jira ticket. Today: `DF-001 — Set up GitHub account and push first commit`. Students learn what a ticket is, what "In Progress" vs "Done" means, and how to move it.

Day output: First commit on GitHub. First PR opened. Jira ticket closed.

Day 2 — JavaScript Fundamentals (Part 1)

Concepts:

- Variables: `let`, `const`, differences
- Data types: string, number, boolean, null, undefined
- Arrays: creating, accessing, `.push()`, `.pop()`, `.length`
- Objects: key-value pairs, dot notation, bracket notation
- Functions: declaration, arguments, return values
- `console.log()` for debugging

What students do:

- Write a Node.js script (`practice.js`) with 10 exercises
- Exercises cover: string manipulation, array filtering, object creation, function writing
- Commit the file and push to their practice repo

Jira: `DF-002 — Complete JavaScript basics exercises and open PR`

Day output: Working JS script committed. PR reviewed by AI (logic correctness, naming, structure).

Day 3 — JavaScript Fundamentals (Part 2)

Concepts:

- Conditionals: `if`, `else if`, `else`, ternary operator
- Loops: `for`, `while`, `for...of`, `for...in`
- Array methods: `.map()`, `.filter()`, `.find()`, `.forEach()`, `.reduce()`
- String methods: `.split()`, `.join()`, `.includes()`, `.trim()`, `.replace()`
- Template literals (backtick strings)
- Scope: what is in scope and what isn't

What students do:

- Extend `practice.js` with 10 more exercises using the above methods

- Write a short script that reads an array of students, filters by score, and prints a formatted summary using template literals

Jira: `DF-003 — JS arrays, loops, and string methods`

Day output: Extended practice script. PR reviewed.

Day 4 — Async JavaScript and APIs

Concepts:

- What an API is: client makes a request, server sends a response
- JSON format: what it looks like, how to read it
- `fetch()` and how it works
- Promises: `.then()`, `.catch()`
- `async/await` syntax
- Reading API documentation
- HTTP methods: GET, POST, difference between them

What students do:

- Write a Node.js script that calls a free public API (e.g. `https://api.coindesk.com/v1/bpi/currentprice.json`)
- Print specific fields from the response in a formatted output
- Handle errors (what happens if the network fails)

Jira: `DF-004 — Make your first API call and handle the response`

Day output: Working API script. Clear understanding of request/response. PR reviewed.

Day 5 — Frontend vs Backend + The Git Branch Workflow

Concepts:

- What frontend means: the browser, HTML/CSS/JS, what the user sees
- What backend means: the server, the database, the logic users never see
- How frontend and backend talk to each other (HTTP, JSON)
- Git branching: what a branch is, why teams use them
- The full PR workflow: `branch → build → commit → push → PR → review → merge`
- Branch naming conventions: `feat/`, `fix/`, `chore/`

What students do:

- Draw (or describe in a `.md` file) how a login flow works: user submits form → frontend sends POST → backend checks DB → returns token → frontend stores it
- Practice the branch workflow: create a feature branch, make a change, open a PR with a proper title and description

Jira: `DF-005 — Model a login flow and practise the full PR workflow`

Day output: Documented flow diagram. PR with proper title, description, and linked Jira ticket. Git branch workflow solid.

Week 1 Summary — What Students Have After Week 1:

- GitHub account with 5+ commits
- 5 merged PRs
- Understanding of terminal, Git branching, JavaScript fundamentals, async/await, API calls
- Familiarity with Jira ticket workflow they'll use every week
- Can describe what frontend and backend mean

Week 2 — Node.js, Express, PostgreSQL & Prisma

Goal: Build a real REST API from scratch. By end of Week 2, students have a working backend with a database — endpoints they can test with Postman or curl.

Day 1 — Node.js and npm

Concepts:

- What Node.js is: JavaScript running outside the browser
- npm: installing packages, `package.json`, `node_modules`
- `require` vs `import`
- Creating a simple Node.js script that runs a web server (http module)
- What `nodemon` does and why it's useful

Day output: Basic Node.js server running on port 3000. Responds to a GET request.

Day 2 — Express and Routes

Concepts:

- What Express is: a framework that simplifies building servers
- Routes: `app.get()`, `app.post()`, `app.put()`, `app.delete()`
- Route parameters: `/users/:id`
- Query parameters: `/users?role=admin`
- `req.body`, `req.params`, `req.query`
- `res.json()`, `res.status()`
- Middleware: what it is, how it runs before your route handler

Day output: Express app with 5 routes (users CRUD). Returns hardcoded JSON for now.

Day 3 — PostgreSQL and Prisma Setup

Concepts:

- What a relational database is: tables, rows, columns, relationships
- PostgreSQL: a production-grade open source database
- Prisma: a type-safe ORM that writes SQL for you
- `schema.prisma`: defining models
- `prisma migrate dev`: pushing schema to DB
- `prisma studio`: viewing data in a GUI
- Connecting Prisma to Express

Day output: Prisma schema with `User` model. Database running locally. Prisma connected to Express app.

Day 4 — Full CRUD with Prisma

Concepts:

- `prisma.user.create()`, `findMany()`, `findUnique()`, `update()`, `delete()`
- Handling not-found errors (return 404, not 500)
- Input validation: checking required fields before hitting the database
- What happens if you try to create a duplicate unique field

Day output: All 5 routes now read/write from PostgreSQL via Prisma. Students can create, read, update, and delete users through the API.

Day 5 — Error Handling and Project Structure

Concepts:

- Centralised error handling in Express
- Separating routes from controllers (folder structure)
- Environment variables: what `.env` is, why secrets don't go in code
- `dotenv` package
- Testing your API with Postman

Day output: Clean project structure (routes/, controllers/, lib/). API fully functional. `.env` file used for `DATABASE_URL`. PR reviewed.

Week 2 Summary — What Students Have After Week 2:

- A working REST API with full CRUD
- PostgreSQL database with a Prisma schema
- Clean folder structure matching industry conventions
- Experience with Postman for API testing
- 10+ merged PRs total

Week 3 — React

Goal: Build a frontend that connects to the backend they built in Week 2. No tutorials. They connect their own API to their own UI.

Day 1 — React Basics and Components

Concepts:

- What React is: a UI library, not a framework
- JSX: writing HTML-like syntax in JS
- Components: functions that return JSX
- Props: passing data into components
- Rendering a list with `.map()`

Day output: Vite + React project. A list of users rendered from hardcoded data.

Day 2 — State with useState

Concepts:

- `useState`: what state is, how it triggers re-renders
- Controlled inputs: keeping form values in state
- Conditional rendering: `{condition && <Component/>}`
- Basic event handling: `onClick`, `onChange`, `onSubmit`

Day output: A form that adds a user to a local state array. No backend yet.

Day 3 — Fetching Data from Their Own API

Concepts:

- `useEffect`: running code after render
- `fetch()` inside React
- Loading states and error states

- Connecting to their own Express API from Week 2

Day output: React app fetches real users from the backend. Renders them. Full-stack connection working.

Day 4 — React Router and Navigation

Concepts:

- React Router v6: `<BrowserRouter>`, `<Routes>`, `<Route>`
- `<Link>` vs `<a>`
- `useParams()`: reading URL parameters
- Multi-page app: home, user list, user detail

Day output: Multi-page React app with navigation. User list page → user detail page.

Day 5 — TanStack Query (React Query)

Concepts:

- Why `useEffect` for data fetching gets messy at scale
- What React Query does: caching, background refetch, loading/error states
- `useQuery`: fetching data
- `useMutation`: creating/updating data
- `queryClient.invalidateQueries()`: refreshing data after a mutation

Day output: All data fetching migrated from `useEffect` to React Query. Create user form uses `useMutation`.

Week 3 Summary — What Students Have After Week 3:

- A working full-stack app: React frontend + Express backend + PostgreSQL
- Understanding of state, props, routing, and data fetching
- TanStack Query for real data management
- 15+ merged PRs total

Week 4 — Authentication + Project 1 Kickoff

Goal: Add JWT authentication to the full-stack app (2–3 days), then get oriented on Project 1 and open the first real project ticket.

Day 1 — Authentication Concepts and Backend Implementation

Concepts:

- What authentication is: verifying who you are
- Passwords and hashing: `bcrypt`, why you never store plain text
- JWT: what a JSON Web Token is, what's inside it, how it's verified
- Access tokens vs refresh tokens: why two tokens, how they differ
- `POST /auth/register` and `POST /auth/login` routes

Day output: Register and login endpoints working. Passwords hashed. JWT returned on login.

Day 2 — Protected Routes and Frontend Auth

Concepts:

- Middleware to verify JWTs: extracting and validating Bearer tokens
- Protected routes: routes that require a valid token
- Storing tokens in the frontend (localStorage, Zustand store)
- Attaching the token to every API request via Axios interceptors
- Redirect unauthenticated users to login

Day output: Protected backend routes. Frontend login flow. Token stored and sent with every request. Redirect on 401.

Day 3 — Refresh Tokens and Role-Based Access

Concepts:

- Refresh token flow: when access token expires, use refresh token to get a new one
- Storing refresh tokens in the database for revocation
- Role-based access: `ADMIN` vs `USER` — checking role in middleware
- Logout: deleting the refresh token from the database

Day output: Complete auth system. Token refresh working. Role guard on admin routes. Logout endpoint.

Day 4 — Project 1 Kickoff

Students receive the Project 1 brief. They read it, break it into Jira tickets, plan their approach, and set up the project repo.

What Project 1 is: A SaaS-style web application with user authentication, data management, file uploads, and a working dashboard. The specific domain is chosen by the student (e.g. invoice manager, student tracker, inventory system) — what matters is the technical requirements, not the theme.

What students do on Day 4:

- Read the Project 1 brief and acceptance criteria
- Create Jira tickets for the first sprint (5–8 tickets)
- Set up a new GitHub repo from the DevForge Project 1 template
- Configure the database schema for their project
- Open the first ticket, create a branch, make the first commit

Day output: Project repo created. Schema designed. First PR opened on the project.

Week 4 Summary — What Students Have After Week 4:

- Complete JWT auth system (register, login, refresh, logout, role-based access)
- Project 1 set up and first PR merged
- 20+ merged PRs total

Week 5 — Project 1: Core Features

Goal: Build the main features of Project 1. By end of Week 5, the app's core CRUD is working and the frontend is connected.

Students work from their own Jira tickets, opening PRs for each feature. AI reviews every PR.

What gets built this week (across their project):

- Database schema finalised and seeded
- Core CRUD API (create, read, update, delete for the main entity)
- React frontend: list view, detail view, create/edit forms
- Auth gates: only logged-in users see the dashboard
- Basic error handling and loading states

No daily breakdown this week — students work from tickets they created. The project brief gives guidance on what must be done by end of Week 5.

Week 5 Summary — What Students Have After Week 5:

- Working full-stack app with auth, CRUD, and a connected React UI
- 30+ merged PRs total
- Experience reading and closing Jira tickets independently

Week 6 — Project 1: File Uploads, Cloudinary & Polish

Goal: Add file upload functionality to Project 1, integrate Cloudinary for storage, and polish the app to a deployable state.

Concepts covered:

- What Cloudinary is: a cloud image/file storage service
- `multer`: handling multipart form data in Express
- Uploading to Cloudinary from Node.js: the `cloudinary` npm package
- Storing the returned Cloudinary URL in the database
- Displaying uploaded images/files in React
- UI polish: empty states, loading spinners, error messages, responsive layout

What students build:

- File/image upload feature on their Project 1 entities (e.g. upload a profile photo, an invoice PDF, a product image)
- Images displayed in the frontend using the Cloudinary URL
- Polished UI with proper error and loading states

Project 1 Deployment (end of Week 6):

- Frontend deployed to Vercel
- Backend deployed (Railway or Render)
- Environment variables configured in deployment platform
- Project 1 link submitted and publicly accessible

Week 6 Summary — What Students Have After Week 6:

- Project 1 fully deployed and live
- Cloudinary file uploads working
- 40+ merged PRs total
- First app in their portfolio

Week 7 — Project 2: Payments with Razorpay

Goal: Build Project 2 — a payment-enabled application using Razorpay. Students learn payment integration in the context of a real product feature, not in isolation.

Concepts covered:

- What Razorpay is and how online payments work in India
- Razorpay orders API: creating an order server-side
- Razorpay checkout widget: embedding payment UI in React
- Verifying payment signatures: why you verify server-side, never client-side
- Webhook basics: what happens when a payment completes asynchronously
- Storing payment records in PostgreSQL

Project 2 brief: A subscription or payment-gated application. Examples: a paid event booking system, a subscription newsletter, a course purchase flow. The student picks a domain; the technical requirements (Razorpay + payment-gated features) are fixed.

What gets built this week:

- Project 2 repo set up, schema designed, first tickets created
- Razorpay order creation endpoint
- Payment checkout flow in React (create order → open Razorpay modal → user pays)
- Payment verification endpoint
- Payment record saved to DB on success
- Feature unlocked after payment (e.g. access granted, seat booked)

Week 7 Summary — What Students Have After Week 7:

- Working Razorpay payment integration
- Project 2 core feature functional
- 48+ merged PRs total

Week 8 — Project 2: Real-time Features with Socket.io

Goal: Add real-time functionality to Project 2 and deploy it.

Concepts covered:

- What real-time means: the server pushes data to the client without a page refresh
- WebSockets vs HTTP: what's different, when to use each
- Socket.io: setting up a server, connecting from a React client
- Emitting and listening to events
- Real-time use cases: live notifications, live order status, chat, live count updates

What gets built this week:

- Socket.io added to the Project 2 Express server
- React frontend connects via Socket.io client

- At least one real-time feature in Project 2 (e.g. payment status updates live without refresh, live seat count on a booking page, basic notification when something changes)

Project 2 Deployment (end of Week 8):

- Full Project 2 deployed (frontend + backend + database)
- Real-time feature working in production
- Project 2 link submitted

Week 8 Summary — What Students Have After Week 8:

- Project 2 fully deployed and live
- Real-time feature via Socket.io
- 56+ merged PRs total
- Two apps in portfolio

Week 9 — Project 3: Production Deployment & DevOps Basics

Goal: Build and deploy Project 3 with proper production practices. Focus on deployment, environment management, and CI/CD basics.

Concepts covered:

- Environment configuration: `.env.development` vs `.env.production`
- What CI/CD means: code pushed → tests run → app deployed automatically
- GitHub Actions: writing a basic workflow file
- Vercel for frontend: automatic deployments on push to main
- Railway / Render for backend: deployment config
- Database migrations in production: running `prisma migrate deploy`
- Environment variables in deployment platforms (not in code)
- Health check endpoints and why they matter
- Reading deployment logs to debug production failures

Project 3 brief: An admin dashboard + public-facing app. Combines everything: auth, CRUD, file uploads, and deployment. The domain is up to the student. Example: a job board with a public listing page and an admin panel to manage listings.

What gets built this week:

- Project 3 repo set up and first tickets created
- Core features built using all previous knowledge
- GitHub Actions CI/CD pipeline: lint + build check on every PR
- Full deployment: frontend on Vercel, backend on Railway/Render

- Production environment variables configured
- Project 3 publicly accessible

Week 9 Summary — What Students Have After Week 9:

- Project 3 fully deployed and live
- CI/CD pipeline running on every PR
- Production deployment workflow understood
- 65+ merged PRs total
- Three apps in portfolio

Week 10 — Career Week

Goal: Prepare students for job applications. Resume, GitHub profile, LinkedIn, mock interview, and application strategy.

Day 1 — GitHub Profile and Project Presentation

What students do:

- Write proper `README.md` files for all three projects (what it is, what it does, tech stack, live link, screenshots)
- Update their GitHub profile README (who they are, what they've built, how to reach them)
- Pin the three DevForge projects on their GitHub profile

Output: GitHub profile that tells a clear story when a recruiter visits it.

Day 2 — Resume

What's covered:

- What technical hiring managers look for in a fresher resume
- How to describe a project in 2 bullet points (what it does + what tech you used)
- What to cut: irrelevant coursework, vague skills lists, "familiar with" wording
- ATS basics: why formatting matters, what keywords to include
- The DevForge resume template walkthrough

What students do:

- Write their resume using the DevForge template and guide
- Self-review using the provided checklist

Output: A completed first draft resume.

Day 3 — LinkedIn and Online Presence

What's covered:

- Why LinkedIn matters for junior developer hiring in India
- Headline: write it as a role, not a title ("Full-Stack Developer | Node.js · React · Open to Work")
- About section: 3–4 sentences, what you've built, what you're looking for
- Projects section: adding the three DevForge projects with live links
- Activity: how to be visible without being annoying

Output: LinkedIn profile updated.

Day 4 — Interview Prep

What's covered:

- What a technical interview for a junior developer role looks like in India
- DSA: the 10 most commonly asked problems for freshers (with worked solutions)
- Project walkthrough: how to explain what you built in 3 minutes
- Behavioral questions: "tell me about yourself", "why this company", "biggest challenge"

Output: Students practice explaining one of their projects out loud (can record a Loom video for self-review).

Day 5 — Job Application Strategy

What's covered:

- Which companies to target as a fresher: early-stage startups vs product companies vs service companies
- Where to find openings: LinkedIn, Naukri, company career pages, referrals
- How many to apply to per week and how to prioritise
- When and how to follow up
- The application tracker spreadsheet

Output: Week 1 application plan ready. First 10 companies identified and added to tracker.

Week 10 Summary — What Students Have After Week 10:

- GitHub profile with 3 live projects and 65+ merged PRs
- Completed resume
- Updated LinkedIn profile
- Job application tracker with first companies identified

- Interview prep done

What Students Graduate With

Item	Details
GitHub profile	3 deployed apps, 65+ merged pull requests
Project 1	Full-stack SaaS app with auth, CRUD, file uploads (Cloudinary)
Project 2	Payment-enabled app with Razorpay + real-time features (Socket.io)
Project 3	Admin + public app with CI/CD pipeline
Resume	Written using DevForge template and guide
LinkedIn	Updated profile with projects and searchable headline
Certificate	Completion certificate with unique verification link
Job readiness	Application tracker, 10 companies identified, interview prep done

---

Jira Workflow — Used Every Week

From Day 1 to Day 5 of Week 10, every piece of work has a Jira ticket.

Ticket lifecycle:

1. Student reads the ticket and understands the acceptance criteria
2. Creates a branch: `feat/DF-001-setup-github` (ticket code in branch name)
3. Builds the feature or completes the task
4. Opens a PR and links the Jira ticket in the description
5. AI reviews the PR
6. Student fixes issues, pushes again, gets re-reviewed
7. PR merged → ticket moved to Done

Why this matters: Every company that uses Jira, Linear, or GitHub Issues runs this workflow. Students finish DevForge already knowing it by muscle memory.

Time Commitment

- **2–3 focused hours per day** (on curriculum weeks)
- **3–4 hours per day** (on project weeks)
- No fixed schedule — fully self-paced
- Can be stretched to 20 weeks with no penalty
- All content, project briefs, and AI PR reviews are available 24/7

Document version: May 2025 · DevForge Cohort 3 · Internal use